ReadFlow — Built Prompt Artifact

Copy-ready prompt for an AI coding agent · Oct 25, 2026

1. Product overview and creative brief

You are building **ReadFlow**, a mobile-first web app that turns text-based PDFs into a calm, book-like reading experience, with a one-tap return to the original PDF. Tagline: *Fall into reading.* This is a working prototype with real PDF handling, not a static design demo. Do not start coding until you have inspected the repository and read this whole brief.

Users: exam aspirants, college students, researchers, professionals, casual readers. Most read on mid-range Android phones, in long sessions, often at night.

Product principle: the interface around the document is expressive; the reading canvas itself is silent. Text never animates, shifts or fades while the person reads.

Chosen creative concept: "Folio"

Three concepts were weighed. One was chosen and must be used everywhere.

Concept	Idea	Verdict
Folio	A bound book and a bookmark ribbon. Warm paper, ink, one terracotta accent, serif voice, hairline rules, page folios.	**Chosen.** Fits a reading product, supports five themes cleanly, stays calm.
Night Study	Cinematic dark lamp-lit desk, glowing surfaces.	Rejected. Dark-first excludes daytime readers and drifts into glow effects.
Pocket Atelier	Tactile tool UI with chunky controls and sticker-like highlights.	Rejected. Playful, but fights long-form reading and ages quickly.

Folio vocabulary (use these, do not invent others):

- **The ribbon**: a thin terracotta bookmark ribbon is the one signature motif. It is the bookmark icon, the reading-progress line, the continue-reading marker, and the first-run mark. Nothing else is decorative.
- **The folio**: small page numbers set in the serif, tabular figures, as quiet marginal labels.
- **The rule**: hairline dividers instead of boxed cards wherever content is a list.

- **The cover**: documents are shown as book covers with a spine edge, not as generic cards.

Voice of the copy: plain, warm, short. No exclamation marks, no emoji, no AI language.

2. Visual direction and design system

Implement these as CSS variables in `src/styles/tokens.css` and expose them through Tailwind v4's `@theme`. Components read tokens only; no hard-coded colors, radii or durations in components.

Color: five reading themes

Set on `<html data-theme="…">`. Values are starting points; adjust only if a contrast check fails (body text 4.5:1 minimum, large text and UI 3:1).

Token	Paper & Ink	Sepia	Forest	Midnight	OLED Black
`--bg` (app and canvas)	#F6F0E4	#EBDDC1	#16241D	#10131C	#000000
`--surface` (sheets, panels)	#FBF7EE	#F2E7CF	#1D2F26	#171B27	#0B0B0B
`--ink` (body text)	#1D1A16	#3B2E1E	#E4EBDD	#E7E4DC	#E8E6E1
`--ink-2` (secondary)	#4A443B	#5A4630	#BCCABB	#B8B5AD	#B5B2AB
`--ink-3` (metadata)	#6B6458	#6A5538	#8FA394	#8D8B86	#8A8780
`--line` (hairlines)	#D9CFBD	#CDB98F	#2F4539	#262B3A	#1E1E1E
`--accent` (ribbon, focus)	#B4482A	#9C3F1E	#E0A458	#E88A63	#E88A63
`--on-accent`	#FFF7EE	#FFF7EE	#16241D	#10131C	#000000

Highlight colors (4, shared across themes, drawn as a background wash behind unchanged ink text): Saffron #F2C14E, Clay #E8A08A, Sage #A9C5A0, Slate #9DBCD4. Light themes: 45% opacity. Dark themes: 32% opacity. Verify ink-on-highlight contrast in every theme and adjust the opacity per theme, not the ink.

State colors: success uses Sage, error uses a deep red (#A33A2C light, #E58A7B dark). Never rely on color alone: pair with an icon and text.

Gradients: none, except a 24px fade at the edges of horizontally scrolling rows. No glass blur anywhere.

Typography

Self-host variable fonts through Fontsource (verify the exact package names and that each font is OFL-licensed before installing). Preload only the reading font and the UI font.

Role	Font	Notes
Brand and editorial headings	Fraunces (variable)	Axes: opsz auto, SOFT 40, WONK 0. Used for the wordmark, library headings, empty states, document titles on covers.
Reading text	Literata (variable)	Designed for long on-screen reading. Default 18px, user range 14 to 28px.
Interface	Instrument Sans (variable)	Buttons, navigation, labels, sheets. 14 or 15px.
Numbers	Fraunces with `font-variant-numeric: tabular-nums lining-nums`	Page numbers, percentages, progress.

Rules: headings are sentence case. Only two heading sizes on any screen. Labels are 12px, weight 500, normal case, tracking 0.01em; do not uppercase labels. Reading column: 62ch maximum, line-height 1.65, paragraph spacing 0.9em, text-align left (optional justify with `hyphens: auto` as a setting, off by default). Provide a fallback stack of `Georgia, serif` and `system-ui, sans-serif` with `font-display: swap` and size-adjusted fallbacks to avoid layout shift.

Space, shape and surfaces

- **Spacing:** 4px base unit; mobile page gutter 20px; minimum touch target 44 by 44px.
- **Radius hierarchy (not one radius for everything):** 2px for inputs and chips; 6px for buttons; covers are 2px on the spine side and 6px on the opposite side; bottom sheets 20px on top corners only; full pill only for the progress thumb and the mode switch indicator.
- **Borders over shadows:** hairline 1px `--line` rules divide lists. Three shadows only, tinted warm in light themes and removed in OLED: `--shadow-1` 0 1px 0 var(--line); `--shadow-2` 0 8px 24px -12px rgba(40,24,8,.28) for covers on hover and floating bars; `--shadow-sheet` 0 -16px 40px -16px rgba(40,24,8,.3).
- **Icons:** Lucide, 20px, stroke width 1.5, one wrapper component that fixes both. No filled icons except the active bookmark ribbon.
- **Buttons:** primary = solid `--ink` with `--bg` text (not accent); accent is reserved for the ribbon, focus rings and the single most important action per screen (Import). Secondary = hairline border. Tertiary = text. Disabled = 45% opacity, no pointer events, and the reason is available as a tooltip or helper line.

- **Focus:** 2px `--accent` outline, 2px offset, visible on keyboard focus only (`:focus-visible`).
- **Selection color:** `--accent` at 25% opacity.

Texture and depth

One optional paper-grain overlay, applied only to the app background (never behind reading text): a 160px tiling SVG noise at 3% opacity, `pointer-events: none` , disabled in OLED and when reduced-motion or lite mode is on. If it reduces legibility in any theme, remove it. Cover thumbnails get a 4px spine gradient on the left edge (one of the few allowed gradients, because it carries meaning: the book edge).

3. Screen-by-screen UI requirements

Routes (use `wouter` , about 2 KB): `/` library, `/read/:id` reader, `/notes/:id` full-screen notebook. Settings, notebook peek, ambience and import live in sheets or panels, not routes.

A. Library (home)

- **Header:** wordmark in Fraunces with the ribbon mark, a search button that expands into an input, a sort control (Recent, Title, Progress), and the Import button (the one accent-colored action).
- **Continue reading:** only when at least one document has progress between 1% and 99%. Shows the most recently opened one as a large cover (about 40% of width on mobile, left aligned), title in Fraunces, "page 42 of 310" with tabular figures, a ribbon progress line, and a "Continue" button. Never invent documents or activity.
- **All documents:** a two-column cover grid on phones, three to five columns on larger screens. Each cover shows the page-1 thumbnail when it renders, otherwise a typographic cover generated deterministically from the title (one of six token-based colorways, title in Fraunces, spine edge). Under the cover: title (two lines max), a hairline, "13% · opened 2 days ago". Long-press or a kebab menu offers Rename and Remove. Remove soft-deletes with a 6-second Undo toast, then purges the file and its annotations.
- **Empty state (first use):** the page's main composition, not a small card. A line-drawn open book with a ribbon (SVG, animated once, see section 4), the heading "Bring a PDF. Fall into reading.", one sentence ("Works best with text-based PDFs. Your files stay on this device."), the Import button, and a drag-and-drop zone on desktop. Beneath it, two small lines explaining Comfort and Original modes. No tour, no carousel.
- **No results state** for search: plain text, a clear-search action.

B. Import flow

- Entry points: the Import button (opens the native file picker with `accept="application/pdf,.pdf"`), drag-and-drop anywhere on desktop (full-window drop overlay with dashed hairline and the text "Drop to add"), and the empty-state zone.
- On selection, open an **import sheet** (bottom sheet on mobile, centered dialog on desktop) showing: file name, size, then a three-step progress list: Reading file, Finding text (page n of N), Making cover. Show a determinate bar using the PDF loading progress callback and extraction page counts. Never fake progress.
- Validate before parsing: check the `%PDF-` header bytes, not only MIME or extension. Handle and explain each failure in plain words with a recovery action: not a PDF; password protected (state that protected files are not supported in this version); corrupted or unreadable; no extractable text (offer to open in Original mode and state that OCR is not supported yet); file over 150 MB (warn, allow); storage full (show remaining quota from `navigator.storage.estimate()`).
- Duplicate detection: hash the file with `crypto.subtle` SHA-256; if it exists, say "Already in your library" and offer Open.
- The reader opens as soon as the first pages are extracted. Extraction continues in the background with a small progress chip in the reader chrome.
- Success: the cover animates from the import sheet into the library grid or straight into the reader (section 4).

C. Reader workspace

Mobile chrome (default hidden after first open; tap the center of the text to toggle):

- Top bar: back, document title (one line, truncated) with "p. 42 of 310", Comfort/Original segmented switch.
- Bottom bar: page scrubber (a slider with a page bubble while dragging), then four icon buttons: Reading settings (Aa), Notebook, Ambience, Bookmark.
- A 2px ribbon progress line sits at the very top edge and stays visible even when the chrome is hidden.
- Chrome never auto-hides while a sheet is open or text is selected. Respect `env(safe-area-inset-*)`.

Desktop: the canvas is centered at the reading measure with generous margins. The toolbar is a slim top bar that fades to 0.0 opacity after 3 seconds of inactivity and returns on pointer movement. Settings and Notebook dock as a 360px panel on the right that pushes the canvas, never overlays the text.

Comfort Reader canvas: single column, user typography settings, a small folio ("42") in the margin at each original page boundary with a hairline rule above it, so position and highlights map to original pages. Extracted headers and footers that repeat across pages are hidden by default, with a visible "Show page furniture" toggle in settings. A page the extractor distrusts (suspected multi-column, tables, mostly numeric or symbol content) shows a thin inline note: "This page may read better in the original" with a Switch link. Never silently reorder or rewrite text.

Original mode: pages rendered on canvas in a vertical scroll, page-fit width by default, pinch and double-tap zoom (1x to 4x), horizontal pan when zoomed, same chrome and scrubber, bookmarks supported. Highlights are Comfort-mode only in this version; say so in the notebook when relevant.

D. Reading settings (sheet on mobile, docked panel on desktop)

One sheet with a live preview: the reader behind it updates immediately, and the sheet is half-height so the text stays visible. Controls, grouped by hairlines, not cards:

1. Theme: five swatches shown as small paper samples with the theme name; the current one has a ribbon corner.
2. Text size: slider 14 to 28px with a large "Aa" at each end and a numeric readout.
3. Font: Literata, plus two additional options only if free and self-hosted (suggest Source Serif 4 and Atkinson Hyperlegible Next). Verify licenses.
4. Line spacing (1.4 to 2.0), paragraph spacing (0.4 to 1.5em), margins (8 to 40px), content width (46 to 72ch).
5. Motion: System / Reduced / Full.
6. Ambient volume lives in the Ambience control, not here.

"Reset to defaults" is a text button at the bottom.

E. Notebook (highlights, notes, bookmarks)

A refined personal notebook, not a CRUD table.

- Two tabs: Highlights and Bookmarks (a segmented control).
- Highlights are listed in document order, grouped under page folios with hairlines. Each entry: the quoted text set in Literata with its color wash behind it, the note beneath in Instrument Sans, tap to jump. A filter row of the four color dots plus "All". Inline note editing (tap "Add note"). Delete with Undo.
- Bookmarks are listed as ribbons: page number, a one-line excerpt, optional label, tap to jump.
- Empty states are specific: "Select any text to highlight it" with a tiny line illustration, and "Tap the ribbon to mark your place".

- On mobile this is a tall bottom sheet; on desktop the docked right panel. `/notes/:id` shows the same content full screen.
- **Creating a highlight:** selecting text in Comfort mode reveals a docked action bar (above the bottom chrome on mobile, floating near the selection on desktop) with four color dots, Note, and Copy. Mobile uses a docked bar, not a floating menu, because native selection handles and callouts collide with floating menus.

F. Ambience

A compact control opened from the bottom bar: three options (Rain, Forest, Seashore) as large touch tiles with a small animated level meter on the active one, one play/pause button, and a volume slider. No autoplay, ever. See section 7 for audio sources.

4. Motion design specification

Principles. Motion explains where something came from or went, confirms an action, or shows progress. It is fast (most things under 240ms), animates only `transform` and `opacity` (plus `clip-path` for the theme reveal), and is absent from the reading text. If you cannot state the purpose of an animation in one sentence, delete it.

Tokens (`src/motion/tokens.ts` and CSS variables, kept in sync)

Token	Value	Use
`--dur-instant`	80ms	press feedback, color swaps
`--dur-fast`	160ms	hover, toggles, menus, tooltips
`--dur-base`	240ms	panels, mode switch, highlight wash
`--dur-slow`	400ms	library to reader, import success
`--dur-theme`	380ms	theme reveal
`--ease-out`	cubic-bezier(.22, 1, .36, 1)	entrances, almost everything
`--ease-in-out`	cubic-bezier(.65, 0, .35, 1)	reveals, moves between two visible states
`--ease-in`	cubic-bezier(.5, 0, .75, 0)	exits (always shorter than the matching entrance)
`spring.sheet`	stiffness 380, damping 36, mass 1	bottom sheets, drag release

Token	Value	Use
`spring.tap`	stiffness 600, damping 32	toggle thumbs, ribbon drop

Exit animations run at about 70% of the entrance duration. Stagger is limited to 30ms steps and at most 6 items; anything beyond appears together.

Rule for choosing the implementation

Use the simplest tool that works, in this order:

1. **CSS** (transitions, keyframes): hover, `:active` press scale, focus rings, color changes, the level meter, the once-only illustration draw, skeleton shimmer.
2. **Motion for React** (`import { motion, AnimatePresence, MotionConfig, LayoutGroup } from "motion/react"`): anything that mounts or unmounts, shared-element and layout transitions, drag and spring release, and shared variants. The app root is wrapped in `<MotionConfig reducedMotion="user">` .
3. **View Transitions API** (`document.startViewTransition`) as progressive enhancement for the theme reveal only.
4. **GSAP**: not used in the MVP. Add it only if a single timeline must orchestrate five or more elements with scrubbing, and only after writing down why Motion cannot do it.

Shared variants live in `src/motion/variants.ts` (`fadeUp` , `sheetUp` , `panelSlide` , `pop` , `listItem`). Components import them; they do not define ad-hoc animation objects.

Required behaviors

Moment	Class	Specification
App entrance	Navigation	Wordmark and header fade up 8px over `--dur-base` ; library content follows with one 30ms stagger of at most 6 covers. No splash screen, no intro longer than 400ms total. Skipped on return visits within the same session.

Moment	Class	Specification
Library to reader	Navigation	The tapped cover has a Motion `layoutId`; it scales and moves into the reader's top-left title area over `--dur-slow` while the library fades to 0 in `--dur-fast`. Reader text fades in only after the cover settles (opacity 0 to 1, 160ms, once). Only the cover uses `layout`; never apply layout animation to the reading content. Fallback without shared element: crossfade 200ms. Back reverses it.
Library to notebook route	Navigation	Slide up 24px with fade, 240ms, `--ease-out`.
Bottom sheets (settings, notebook, import, ambience)	Component	Radix Dialog for focus and semantics, wrapped by a custom `Sheet` using Motion drag on the y axis, `spring.sheet`, backdrop opacity 0 to 0.32. Dismiss on drag past 30% of height or velocity above 500px/s. Handle bar 36 by 4px. Desktop panel: slides in from the right 24px with fade, canvas shifts using a single CSS transform on the canvas wrapper, not per-element layout.
Reader chrome	Component	Toggle with fade and 8px translate, `--dur-fast`. Top and bottom bars move opposite directions. Pointer and focus events stay reachable only when visible.
Button press	Micro	`:active` scale .97 in `--dur-instant`, release `--dur-fast`. CSS only.
Toggle (Switch)	Micro	Thumb moves on `spring.tap`; track color crossfades in `--dur-fast`.
Slider	Micro	Thumb scales to 1.15 and shows a value bubble while pressed; the bubble fades in `--dur-instant`. The text preview updates every frame without transitions.

Moment	Class	Specification
Mode switch (Comfort/Original)	Content	Segmented indicator slides with a shared `layoutId`. Content crossfades over `--dur-base` with a 4px vertical offset; the target scroll position is restored before the fade-in so there is no visible jump.
Theme change	Content	Where supported, a circular `clip-path` reveal from the tapped swatch over `--dur-theme` using `startViewTransition`. Otherwise a 200ms transition limited to `background-color`, `color`, `border-color` applied via a temporary class on `<html>` and removed afterward. Inline script in `index.html` sets `data-theme` from `localStorage` before first paint so no flash occurs on load.
Library cover hover	Micro	Only inside `@media (hover: hover) and (pointer: fine)`: lift 3px, spine edge brightens, `--shadow-2` fades in over `--dur-fast`. Touch devices get press scale only.
Import progress	Progress	Bar fills by real values using `transform: scaleX`. Step list: current step shows a small ribbon-colored dot; completed steps show a check that draws in 160ms. Never fake progress or animate toward an unknown total.
Import success	Progress	Check mark, then the new cover flies into place using the same `layoutId` pattern as opening a document.
Import error	Feedback	Sheet content crossfades to the error message; the icon nudges 4px horizontally twice in 240ms (skipped under reduced motion). Message and action are announced via `role="alert"`.

Moment	Class	Specification
Reading progress	Progress	A 2px ribbon line along the top edge. Driven by `useScroll` on the reader container and a `scaleX` transform on a MotionValue, so scrolling never triggers a React render. Page scrubber bubble appears only while dragging.
Bookmark	Feedback	The ribbon icon fills with a short downward spring drop (translateY -6px to 0, `spring.tap`); a toast with Undo appears. Removal reverses it without a toast.
Highlight creation	Feedback	The color wash grows left to right behind the text using a CSS `background-size` transition over `--dur-base`, once. The action bar slides 8px and fades out after saving. Deleting fades the wash out over `--dur-fast`.
Jump to passage	Feedback	After scrolling (instant, never smooth-scrolling a long distance), the target highlight or folio pulses once: wash opacity 1 to .5 to 1 over 600ms.
Soundscape select	Micro	The chosen tile's border moves via `layoutId`; the level meter (three bars, CSS `scaleY` keyframes) runs only while audio plays and stops on pause. Volume change shows no animation beyond the slider.
Empty states	Ambient	Library empty: the open-book line drawing strokes in once over 900ms with CSS `stroke-dashoffset`, then the ribbon drops with `spring.tap`. Notebook empty: a short single stroke-in. No loops.
Toasts	Feedback	Slide up 12px with fade, 3 to 5s, swipe to dismiss, polite live region. Errors persist until dismissed.

Moment	Class	Specification
Skeletons	Progress	Static placeholder blocks for not-yet-extracted pages, with a 1.4s opacity pulse between .55 and .85 at most. Pulse stops under reduced motion.

Hard limits

- Reading text never animates, reflows from animation, or changes opacity while scrolling or settling.
- No `filter: blur`, `backdrop-filter`, or animated box-shadow on elements over scrolling content. Animate a pre-rendered pseudo-element's opacity instead.
- No JavaScript animation loop for decoration. Scroll-linked work uses MotionValues or CSS, never state.
- Animate `will-change` only on elements currently animating; remove after.
- **Reduced motion** (OS setting or in-app Reduced): remove transforms and spatial movement; keep opacity fades at 120ms or less; disable the theme reveal, paper grain, skeleton pulse, level meter bars (replace with a static "playing" dot), stroke-in and ribbon drop.
- **Lite mode:** if `navigator.hardwareConcurrency <= 4` or `navigator.deviceMemory <= 4`, default to reduced layout animation (no shared-element transition, no paper grain). The user can override in Settings > Motion.

5. Researched package recommendations

Checked on 25 Oct 2026 against official docs and READMEs (listed under Sources). **Before installing anything, run `npm view <package> version` and `npm ls`, record the versions you actually install in `DEPENDENCIES.md`, and re-check peer dependencies.** Package names below were verified in documentation except where marked *verify*.

Motion and transitions

Library	Verdict	Reasoning
Motion for React (`motion`, import from `motion/react`)	**Primary. Essential.**	MIT licensed. The successor to Framer Motion: import from `motion/react`, not `framer-motion`. Needs React 18.2 or newer. Gives layout and shared-element transitions, `AnimatePresence`, drag with spring release, `MotionConfig reducedMotion`, and scroll MotionValues, which together cover every required behavior.
GSAP	**Excluded from MVP.**	Now free for commercial use including former paid plugins, so cost is not the reason. It overlaps Motion, adds a second animation engine and imperative lifecycle management with React, and no required behavior needs timeline scrubbing. Revisit only per the rule in section 4.
AutoAnimate	**Excluded.**	Convenient for list changes, but Motion's `layout` and `AnimatePresence` already cover library and notebook lists and keep one engine and one set of tokens.
React Transition Group	**Excluded.**	Redundant with `AnimatePresence`.
CSS transitions and keyframes	**Used heavily.**	First choice for hover, press, focus, meters and the once-only illustration draw.

Component primitives

Library	Verdict	Reasoning
Radix UI via shadcn/ui	**Primary. Essential.**	shadcn components are copied into the repo, so they can be fully restyled to the tokens; they sit on accessible Radix primitives (Dialog, Slider, Switch, ToggleGroup, DropdownMenu, Tooltip). The shadcn CLI supports Vite and Tailwind v4. Install only the components needed and rewrite their classes to Folio tokens. Shipping the default shadcn look is a failure.
Base UI	Acceptable alternative, not combined.	shadcn can target it. Choose Radix or Base UI, never both. Radix is the default for ecosystem maturity.
React Aria Components	**Excluded.**	Excellent touch behavior, but it adds a second styling and primitive model beside shadcn.
Headless UI	**Excluded.**	Smaller set; overlaps Radix.

Styling and tokens

Tailwind CSS v4 with CSS variables. Use the `@tailwindcss/vite` plugin (no PostCSS config, no `tailwind.config.js`; tokens go in `@theme` in CSS). Tokens are CSS variables so themes switch by attribute. No CSS Modules, no CSS-in-JS. If shadcn's init adds `tw-animate-css`, keep it only if a component needs it.

Icons

`lucide-react`: one coherent stroke family, tree-shakeable. Wrap in a single `Icon` component fixing size 20 and stroke 1.5.

PDF

Library	Verdict	Reasoning
`pdfjs-dist` (Mozilla PDF.js)	**Essential.**	Used directly for text extraction (`getTextContent`) and canvas rendering. Configure the worker with Vite's URL import: `import workerSrc from "pdfjs-dist/build/pdf.worker.min.mjs?url"` and set `GlobalWorkerOptions.workerSrc`; add `optimizeDeps.include: ["pdfjs-dist"]` if Vite pre-bundling causes errors; add a `declare module` for the `?url` import in TypeScript. If target browsers lack newer JS features (such as `Promise.withResolvers`), use the legacy build (`pdfjs-dist/legacy/build/...`) for both library and worker. Copy `cmaps` and `standard_fonts` to `public/` and pass `cMapUrl` and `standardFontDataUrl` so CJK and non-embedded fonts render without network calls.
`react-pdf` (wrapper)	**Excluded.**	It is a viewer wrapper; ReadFlow needs custom extraction, cancellation and rendering control.

Persistence, virtualization and small helpers

Package	Verdict	Reasoning
`dexie` and **`dexie-react-hooks`**	**Essential.**	A thin, well-maintained IndexedDB wrapper (documented at version 4.x). `useLiveQuery` makes the library and notebook update reactively with no extra state store. Note: wrap non-Dexie calls inside queries in `Promise.resolve()`. Store PDFs as Blobs.
`@tanstack/react-virtual` (*verify*)	**Essential for Comfort mode.**	Virtualizes pages and blocks with dynamic measurement, so a 500-page book never mounts more than a window of text.

Package	Verdict	Reasoning
`sonner` (*verify*)	**Recommended.**	Small toast system with swipe dismissal, themed via CSS variables. If it cannot match the tokens, use shadcn's Radix toast instead.
`wouter` (*verify*)	**Recommended.**	Roughly 2 KB router, enough for three routes, works with `AnimatePresence`.
`clsx`, `tailwind-merge`, `class-variance-authority`	**Essential** (installed by shadcn).	Class composition for components.
Fontsource variable packages for Fraunces, Literata, Instrument Sans (*verify names and licenses*)	**Essential.**	Self-hosted fonts, no third-party font requests.
Web Audio API	**Native, no package.**	Gain control and gapless looping with `AudioBufferSourceNode`.
Gesture library (`@use-gesture/react`)	**Excluded.**	Pinch-zoom is written with pointer events; sheet dragging uses Motion drag.
State library (Zustand, Redux)	**Excluded.**	React context for UI state and Dexie live queries for persisted data are enough.
Audio libraries (Howler.js, Tone.js)	**Excluded.**	Unneeded weight for three looping beds.
Lottie, three.js, particle libraries	**Excluded.**	Decorative, heavy, against the art direction.

6. Dependency and architecture plan

First step of every session: inspect `package.json`, lockfile, `vite.config.*`, `tsconfig*`, `src/`, and any README. Reuse what exists. Install only what this plan names, and only if it is not already present. Use TypeScript.

Final stack

1. **Core:** `react`, `react-dom`, `vite`, `@vitejs/plugin-react`, `typescript`, `pdfjs-dist`, `dexie`, `dexie-react-hooks`, `@tanstack/react-virtual`, `wouter`.
2. **UI and motion:** `tailwindcss`, `@tailwindcss/vite`, shadcn/ui components (Radix primitives: dialog, slider, switch, toggle-group, dropdown-menu, tooltip), `motion`, `lucide-react`, `clsx`, `tailwind-merge`, `class-variance-authority`, Fontsource variable fonts (Fraunces, Literata, Instrument Sans), `sonner`.
3. **Optional enhancements** (add only after the MVP works and only with a measured benefit): `vite-plugin-pwa` for offline shell and installability; a Markdown export of notes (no package needed); `@axe-core/playwright` and `vitest`, `@testing-library/react`, `playwright` as dev dependencies for testing.
4. **Explicitly excluded from the MVP:** GSAP, AutoAnimate, React Transition Group, React Aria, Headless UI, Base UI (unless replacing Radix wholesale), `react-pdf`, Zustand or Redux, `@use-gesture/react`, Howler, Tone.js, Lottie, three.js, any particle or canvas background library, any OCR or AI package, any backend, auth or payment SDK.

Module layout

```
src/
  app/            shell, providers (MotionConfig, theme, settings), routes
  styles/         tokens.css, base.css, reader.css
  motion/         tokens.ts, variants.ts, Sheet.tsx, useReducedMotion helper
  ui/             restyled shadcn primitives, Icon, Cover, Ribbon, Toast host
  features/
    library/      LibraryPage, ContinueReading, CoverGrid, EmptyState,
search/sort
    import/       ImportSheet, validateFile, hashFile, import pipeline
    pdf/          pdfClient (load, worker, destroy), renderPage, thumbnail
    extract/      extractPage, groupLines, buildBlocks, furniture detection,
quality hints
    reader/       ReaderPage, ComfortView, OriginalView, Chrome, Scrubber,
ModeSwitch
    settings/     ReadingSettingsSheet, previews, theme reveal
    notes/        NotebookPanel, HighlightBar, anchoring, BookmarkList
    ambience/     AmbienceControl, engine (Web Audio), synth beds
  data/           db.ts (Dexie schema), repositories, migrations
  lib/            hashing, formatting, device capability detection
```

Each feature exports a small public surface; features do not import each other's internals. Motion tokens and variants are shared; no feature defines its own durations.

Data model (Dexie, version 1)

Table	Key fields	Notes
`documents`	id, title, fileName, size, hash, pageCount, addedAt, lastOpenedAt, extractedPages, status, coverBlobId	Metadata only, small, always live-queried.
`files`	docId, blob	The PDF Blob. Loaded only when opening.
`covers`	docId, blob	Page-1 thumbnail, about 360px wide, JPEG or WebP.
`pages`	[docId+pageIndex], blocks, charCount, quality	Extracted text blocks cached per page, so reopening is instant and extraction can resume.
`highlights`	id, docId, pageIndex, blockIndex, start, end, quote, prefix, suffix, color, note, createdAt	Anchored by position plus a text quote for re-anchoring.
`bookmarks`	id, docId, pageIndex, mode, label, excerpt, createdAt	
`progress`	docId, pageIndex, blockIndex, scrollOffset, mode, updatedAt	One row per document.
`settings`	key, value	Typography, theme, motion, ambience. The theme also mirrors to `localStorage` for the flash-free inline script.

Request `navigator.storage.persist()` after the first successful import and handle quota errors.

PDF lifecycle rules

- One `PDFDocumentProxy` per open reader. Call `destroy()` on unmount and when switching documents. Revoke every object URL you create.
- Original mode renders a page only when it is within about one viewport of the screen, cancels the `RenderTask` when it leaves, caps device pixel ratio at 2, and releases canvases for pages far off screen (set `width = 0`, call `page.cleanup()`).
- Extraction runs page by page in order, yielding to the main thread between pages, saving each page to `pages`, and prioritizing the page the reader needs next. Pages the person reaches before extraction finishes are extracted on demand.

7. Functional requirements

Every control in the UI must work. If a feature is not built, the control is not shown.

Import and library

- Import text-based PDFs from the device; show file name, size and page count; handle every error listed in section 3B.
- Library shows only real documents. Search filters by title; sort by Recent, Title, Progress. Rename, Remove with Undo.
- Out of scope and not implemented: OCR (printed or handwritten), AI summaries, question generation, document chat, cloud sync, payments, authentication, native packaging, advanced conversion, 3D environments. Do not add placeholder buttons for them.

Comfort Reader: extraction rules

Extraction must preserve the author's words. The goal is a sensible reading order, not rewriting.

1. Read `getTextContent()` items per page and group them into lines by baseline `y` (tolerance about 0.3 of the median font height), sort lines top to bottom and items left to right.
2. Detect columns by clustering line `x` starts; if two or more stable columns exist, read the left column fully before the right, and flag the page with a quality hint. If detection is uncertain, keep the natural order and flag it.
3. Build paragraphs from lines: a new paragraph starts when the vertical gap exceeds about 1.4 times the median line gap, when a line is indented relative to the body, or when the previous line ended a sentence short of the line width. Detect headings by font size above the median and short length; render them as headings but keep their text unchanged.
4. Hyphenation: join a word split by a trailing hyphen at a line end only when the joined word is lowercase and both fragments are alphabetic. This is the only automatic text change. It is on by default and a visible "Join split words" toggle in settings turns it off.
5. Hide repeated headers, footers and bare page numbers only when the same normalized line appears at the same position on at least 40% of pages. Keep them stored; the "Show page furniture" toggle restores them.
6. Preserve a page marker for every original page (the folio) so progress, bookmarks and highlights map to original page numbers.

7. Score each page's reliability (character count, ratio of non-alphabetic characters, short-line ratio, column uncertainty). Low scores show the "may read better in the original" note. If the whole document has fewer than about 30 characters per page on average, treat it as scanned: explain and offer Original mode. Do not attempt OCR.
8. Tables, formulas and diagrams are not reconstructed. Where extraction finds them likely (many short numeric lines, math symbols, or an image-heavy page), show the inline note and a Switch to original link.

Comfort Reader: rendering

- Virtualize with `@tanstack/react-virtual` using dynamic measurement over blocks grouped per page. Reserve an estimated height for unextracted pages (a static skeleton) so the scrollbar and scrubber are stable, and correct the estimate as pages are measured, preserving the visible anchor to avoid jumps.
- Typography settings apply through CSS variables on the reader container (`--read-size`, `--read-lh`, `--read-para`, `--read-margin`, `--read-measure`, `--read-font`). Changing them must not re-extract text or re-mount pages. When settings change, keep the first visible block pinned in place.
- No horizontal overflow at any size from 320px wide. Long words and URLs break (`overflow-wrap: anywhere`).

Original PDF mode

- Render pages to canvas on demand (section 6 lifecycle), crisp at device pixel ratio up to 2.
- Zoom: pinch and double-tap from 1x to 4x implemented with pointer events. During the gesture apply a CSS transform to the visible page container; when the gesture ends, re-render visible pages at the new scale. Pan horizontally when zoomed. Keyboard: `+`, `-`, `0` on desktop.
- Scrubber and page jump work identically in both modes, and switching modes keeps the same page.

Themes, typography, preferences

Implement all five themes from section 2 (Paper & Ink, Sepia, Forest, Midnight, OLED Black), all typography settings from section 3D, and persist them. Apply the saved theme before first paint.

Highlights, notes, bookmarks

- Highlights exist in Comfort mode. Selecting text across multiple blocks is allowed within one page; if a selection spans pages, split it into one highlight per page.

- Anchor each highlight with `pageIndex`, `blockIndex`, character `start` and `end` within the block's text, and the `quote` plus 24 characters of `prefix` and `suffix`. On load, apply by position; if the text at that position differs (for example after a setting changes joining), re-find the quote using the prefix and suffix; if it cannot be found, keep the entry in the notebook and mark it "Unplaced" rather than deleting it.
- Four colors, optional note, delete with Undo. Tapping a highlight in the text opens a small action bar (change color, edit note, delete).
- Bookmarks work in both modes, store page and mode, and have optional labels.
- Jumping to a highlight or bookmark switches mode if needed, scrolls instantly, then plays the pulse from section 4.
- All annotation data persists in IndexedDB and survives reload.

Reading progress

- Save position (page, block, offset, mode) debounced to at most once per second and on `visibilitychange` and `pagehide`.
- Restore it exactly on reopen, before the first visible paint of the reader content.
- Show percentage as `(pageIndex + fraction) / pageCount`, with tabular figures, in the library and the reader.

Ambient audio

- Three beds: Rain, Forest, Seashore. Play and pause, volume, remembered choice and volume.
- **Sources, in order of preference:** (1) audio files in `public/audio/` that are CC0 or otherwise clearly licensed for this use, with a `public/audio/LICENSES.md` recording title, author, source URL and license for each; do not download or include any file whose license you cannot confirm. (2) If no licensed files are available, use synthesized beds built with the Web Audio API, and label them honestly in the UI and README as synthesized: Rain = band-passed pink noise with sparse short high-frequency noise ticks; Seashore = low-passed brown noise with a slow 0.08 to 0.12 Hz amplitude and cutoff swell; Forest = soft filtered wind noise with occasional quiet sine-sweep chirps at random intervals of 4 to 12 seconds. Synthesized beds must loop without clicks and never exceed a -12 dB headroom at full volume.
- Create the `AudioContext` only on the first user tap. Resume it on user gesture. Fade in and out over 400ms with gain ramps to avoid pops. Use `AudioBufferSourceNode` with `loop = true` for gapless file beds. Pause and release on unmount; suspend when the tab is hidden for more than 60 seconds if playing, and show the paused state accurately on return.

- The whole app works with audio unavailable; if neither files nor Web Audio exist, hide the Ambience control rather than disable it.

Privacy

No network calls with document data. No analytics. No external font or script hosts. State "Your files stay on this device" in the empty state and verify it is true.

8. Mobile-first and responsive requirements

Design at 360 by 800 first, then enlarge. Mobile layouts are composed for the phone, not shrunk desktop pages.

Breakpoint	Layout
Under 640px (phone)	Library: continue-reading cover left with text right, then a two-column grid. Reader: full-bleed canvas, top and bottom chrome, sheets from the bottom.
640 to 1023px (tablet)	Three-column grid. Reader canvas centered at the chosen measure; sheets become centered dialogs up to 480px wide.
1024px and above (desktop)	Library: continue-reading takes five of twelve columns beside a four-column grid. Reader: centered column, slim fading toolbar, settings and notebook as a 360px docked right panel. Keyboard shortcuts enabled.

Rules

- Use `100dvh` (not `100vh`) and `env(safe-area-inset-*)`; set `viewport-fit=cover`, `theme-color` per theme, and `color-scheme` accordingly.
- Primary reading actions sit in the lower half of the screen within thumb reach. Nothing essential is only at the top.
- Touch targets at least 44 by 44px with at least 8px between them. Use `touch-action: manipulation` on controls to remove tap delay, and `overscroll-behavior: contain` inside sheets.
- Support portrait and landscape; in landscape phones, keep the reader chrome hidden by default and the bars compact.
- Text size changes with the OS font setting must not break layouts; use `rem` for UI sizes and the reader's own `--read-size` for content.

- Hover effects exist only inside `@media (hover: hover)`.
- Add a web app manifest and icons so the app can be installed to the home screen; an offline service worker is optional (section 6).

9. Performance and accessibility

Performance budgets (measure, then report the real numbers)

Metric	Target	How to measure
Initial JS, gzip, library route	180 KB or less excluding the PDF.js worker and fonts	`vite build` output and a bundle visualizer
PDF.js	Loaded only when a document is imported or opened (dynamic `import()`)	Network panel on first load
Largest Contentful Paint, mobile profile	Under 2.5s on the library	Lighthouse mobile, throttled
Interaction latency	INP under 200ms for settings changes and mode switch	Chrome DevTools Performance
Scroll, Comfort mode, 300-page fixture	No long tasks over 50ms while scrolling; frame rate 55fps or better at 4x CPU throttling	Performance panel, 6 seconds of continuous scroll
Memory, 300-page document in Original mode	Stays flat after 60 seconds of scrolling; canvases for far pages are released	Performance monitor
Layout shift	CLS under 0.05 on library and reader	Lighthouse and Layout Shift regions

Techniques required: dynamic imports for the reader, PDF.js and the notebook; font preloading with size-adjusted fallbacks; `content-visibility: auto` for notebook entries; passive scroll listeners; `ResizeObserver` and `IntersectionObserver` instead of scroll polling; one MotionValue-driven progress bar; extraction yielding between pages; hashing and extraction never block a frame for more than 50ms (chunk work).

Do not claim animations are smooth without a recorded measurement. Put the measurements in `PERFORMANCE.md`.

Accessibility

- Contrast: verify every theme with a script or tool; body text at least 4.5:1, metadata and UI components at least 3:1; highlight washes keep body text at 4.5:1.
- Keyboard: every control reachable and operable. Visible `:focus-visible` ring. Sheets trap focus, close on Esc, and return focus to the opener. Shortcuts on desktop: `Esc` close, `b` bookmark, `m` toggle mode, `,` open settings, `[` and `]` change text size, arrow keys or Page Up/Down scroll. Show a shortcut list from the settings panel.
- Semantics: landmarks (`header`, `main`, `nav`), the reader canvas is an `article` with a label; headings extracted from the PDF are real headings; the progress line has `role="progressbar"` with value text "Page 42 of 310"; the scrubber is a slider with `aria-valuetext`; toasts use polite live regions and errors use `role="alert"`.
- Do not communicate with color alone (themes, highlight colors need names exposed to assistive tech: "Saffron highlight").
- Support `prefers-reduced-motion`, `prefers-color-scheme` (map to the first-run theme only), `forced-colors` (keep text and focus visible) and zoom to 200% without loss of content.
- Touch alternatives for every gesture: pinch zoom has +/- buttons; sheet swipe has a close button; long-press menus have a visible kebab.

Cleanup and robustness

Remove event listeners, observers and timers on unmount. Revoke object URLs. Destroy PDF documents and cancel render tasks. Close the `AudioContext` source nodes when audio is paused for good. Guard all `localStorage`, `IndexedDB` and `navigator.storage` calls with try/catch and show a plain message if storage is blocked (for example private browsing).

10. Implementation phases

Work in this order. At the end of each phase, run the build, test the phase's exit check, commit, and write two lines in `PROGRESS.md` (what works, what is deferred). Do not start the next phase with a broken build.

Phase	Build	Exit check
0. Inspect	Read the repo, `package.json`, configs. Write `DEPENDENCIES.md` with what exists and what you will add, with versions from `npm view`.	Plan matches section 6; nothing installed that is excluded.

Phase	Build	Exit check
1. Design system and shell	`tokens.css` with all five themes, fonts, Tailwind v4 `@theme`, restyled shadcn primitives, `Icon`, `Ribbon`, `Cover`, `Sheet`, motion tokens and variants, `MotionConfig`, theme persistence with the pre-paint script, responsive shell and routes. A temporary internal `/kit` route shows every token and component in all five themes (remove or hide before finishing).	Contrast script passes for all themes; `/kit` looks coherent at 360px and 1280px; no default shadcn look remains.
2. Library and import	Dexie schema, import pipeline with validation, hashing and duplicate detection, progress sheet, library grid, search, sort, rename, soft-delete, empty state, covers.	A real PDF imports and appears; each error path shows its message.
3. PDF.js and Original viewer	Worker config, `pdfClient`, on-demand canvas rendering, cancellation, zoom and pan, scrubber, lifecycle cleanup.	300-page fixture scrolls with flat memory.
4. Extraction and Comfort Reader	Line and paragraph grouping, columns, furniture hiding, quality scoring, per-page cache, virtualized reader, mode switch.	Reading order is sensible on the fixtures; unusual pages show the note; no text is altered beyond rule 4.
5. Themes and typography controls	Settings sheet with live preview, all controls, persistence, theme reveal.	Changing any setting keeps the first visible block in place.
6. Highlights, bookmarks, persistence	Selection bar, anchoring and re-anchoring, notebook panel and route, bookmarks, jump-to with pulse, progress save and restore.	Reload restores position and all annotations; "Unplaced" path works.
7. Ambient audio	Engine, licensed or synthesized beds, controls.	No autoplay; first tap starts audio; pausing and unmounting leaves no running nodes.

Phase	Build	Exit check
8. Motion pass	Apply every row of the section 4 table using shared tokens; remove any animation lacking a purpose; implement reduced-motion and lite mode.	Each required behavior verified by hand in normal and reduced motion.
9. Hardening	Responsiveness, accessibility, performance measurement, error handling, README.	Section 12 acceptance criteria all pass.

If a package is incompatible or unneeded, choose the simplest maintainable alternative and record the decision and reason in `DEPENDENCIES.md`. Never silently swap the stack.

11. Testing instructions

Fixtures live only under `tests/fixtures/` and are never shown in the app. Generate them with a script (for example `pdf-lib` as a dev dependency): a 3-page simple text PDF; a 300-page text PDF; a two-column PDF; a PDF with repeating headers and footers and hyphenated line endings; an image-only PDF; a password-protected PDF; a truncated PDF; and a non-PDF file renamed `.pdf`. Do not commit copyrighted documents.

Unit tests (Vitest): line grouping, column detection, paragraph building, hyphen joining (and that it changes nothing else), header and footer detection, page quality scoring, highlight anchoring and re-anchoring, progress percentage, file validation, file hashing.

End-to-end (Playwright, mobile Chromium profile plus one desktop):

1. First run shows the empty state; import the 3-page PDF; it appears with correct page count.
2. Each failure fixture shows the specified message and a working recovery action.
3. Open the document; switch Comfort and Original; the page stays the same.
4. Change every typography setting; text updates live; reload; settings persist.
5. Select text, highlight in each color, add a note, reload, confirm it persists and jumps correctly; delete with Undo.
6. Add and remove a bookmark in both modes; jump to it.
7. Scroll, close, reopen: position restores.
8. Start and pause each ambience option; volume changes; confirm nothing plays before a tap.
9. Repeat the main flow with reduced motion emulated; no spatial animation occurs.
10. Open and close every sheet with keyboard only.

Accessibility: run `@axe-core/playwright` on library, reader, each sheet and notebook in every theme with zero serious or critical violations; verify contrast numbers with a script; do a manual screen-reader pass of the import and reader flows (record findings in `A11Y.md`).

Performance: follow section 9. Run Lighthouse mobile on the library, and the Performance panel on the 300-page fixture at 4x CPU throttle in both modes. Record numbers in `PERFORMANCE.md`. If a budget is missed, fix it or record why.

Visual review: capture screenshots of library (empty and populated), import sheet, Comfort reader, Original reader, settings, notebook, ambience in all five themes at 360px and 1280px. Compare against section 13 and fix anything that looks generic.

Before declaring completion: run `npm run build` and `npm run preview`, test the production build on a phone or an emulated one, and list anything not tested.

12. Acceptance criteria

The work is done only when every item is true and demonstrated.

Function

- [ ] A text PDF imports from a phone file picker and by drag and drop on desktop, with real progress, file name, size and page count.
- [ ] All eight failure fixtures produce the specified message and recovery action; none crash or hang.
- [ ] Comfort mode shows single-column text with sensible order on the simple, two-column and header/footer fixtures; no text changed except optional split-word joining.
- [ ] Original mode renders pages, zooms 1x to 4x, pans, and keeps memory flat on the 300-page fixture.
- [ ] Five themes work in both modes and apply before first paint on reload with no flash.
- [ ] Size, font, line spacing, paragraph spacing, margins, width and motion settings update the reader live and persist.
- [ ] Highlights in four colors with notes, and bookmarks, persist across reload, re-anchor correctly, and jump with the pulse; unplaceable highlights are kept as "Unplaced".
- [ ] Reading position restores exactly on reopen; percentages are correct in library and reader.
- [ ] Audio never plays before a tap; play, pause and volume work; the app is fully usable with audio unavailable; any asset has a recorded license, or the bed is labeled synthesized.
- [ ] No control is a placeholder; no fake documents, metrics or activity appear anywhere.

Design

- [ ] A reviewer can name the Folio concept from any screen: ribbon, folio numerals, hairline rules, spine-edge covers.
- [ ] Typography follows section 2; the reading canvas looks different from the surrounding controls; there are no uniform rounded-card grids.
- [ ] No glassmorphism, decorative gradients, floating blobs, emoji, or stock-dashboard patterns.
- [ ] Mobile layouts were designed at 360px, not scaled down from desktop; tablet and desktop use the layouts in section 8.

Motion

- [ ] Every row of the section 4 table is implemented and uses shared tokens and variants; no ad-hoc durations.
- [ ] Reading text never moves while reading; no animation touches properties other than `transform`, `opacity`, `clip-path`.
- [ ] Reduced motion and lite mode behave exactly as specified.

Quality

- [ ] `npm run build` passes with no type errors and no console errors on the main flows.
- [ ] Axe reports zero serious or critical issues in all themes; contrast numbers meet section 9.
- [ ] Performance numbers are recorded in `PERFORMANCE.md` and meet section 9 budgets, or each miss is explained.
- [ ] Initial JS and PDF.js loading follow section 9.
- [ ] Listeners, observers, object URLs, PDF documents, render tasks and audio nodes are cleaned up.
- [ ] `README.md` explains how to run, build and test, the stack decisions, known limits (no OCR, highlights are Comfort-only, protected PDFs unsupported), and the audio licensing or synthesis note.

13. Avoiding the generic AI-generated look

These are binding rules, not suggestions. Before building each screen, write one sentence in `DESIGN_NOTES.md` stating its composition and why. Review every screen against this list before moving on.

1. **Art direction first.** Phase 1 completes the Folio system before any feature screen. Build features from the tokens, never from library defaults.

2. **One accent, one motif.** Terracotta (or amber on dark themes) appears only on the ribbon, focus rings, and the single primary action. Not on icons, headings, or backgrounds in bulk.
3. **Vary form by function.** Covers look like books. Lists are hairline-ruled rows. The notebook looks like a notebook. Settings are grouped by rules, not boxed cards. Floating bars look different from sheets. No two different functions share the same rounded rectangle.
4. **Typographic hierarchy does the work.** Fraunces for voice, Literata for reading, Instrument Sans for tools. Two heading sizes per screen. Whitespace and alignment create grouping, not borders and shadows.
5. **Composed screens.** The library has an asymmetric hero (continue-reading cover left, text right) and a grid with consistent baselines. The empty state is a full composition with an illustration, not a gray box with an icon.
6. **Designed mobile.** Thumb-reach controls, bottom sheets, docked selection bar, hidden-by-default chrome. If a mobile screen looks like a compressed desktop one, redo it.
7. **No fabrication.** No sample documents, no fake recent activity, no invented stats, no decorative charts, no testimonial or marketing sections.
8. **No effect for its own sake.** No glassmorphism, no gradient backgrounds, no floating blobs, no particles, no 3D, no shine or glow, no emoji, no confetti, no gratuitous parallax.
9. **Restraint in shape.** Follow the radius hierarchy; no pill buttons except where specified; no stacked shadows; no nested cards.
10. **Icons are quiet.** One family, one stroke, only where an icon aids recognition; text labels on anything ambiguous.
11. **Every component has a rationale.** In `DESIGN_NOTES.md`, one line per major component: what it is for, and why it looks as it does.
12. **Copy is human.** Short, specific, calm. Avoid "Unlock", "Supercharge", "AI-powered", "Seamless", and exclamation marks.
13. **Use references as inspiration only.** Do not copy another product's layout, logo, name or illustration.

If a screen could belong to any other SaaS product with the logo swapped, it fails.

Sources

Verified on 25 Oct 2026. Fontsource package names, `sonner`, `wouter` and `@tanstack/react-virtual` were not verified in this research and are marked *verify* above.

- Motion for React: installation and requirements

- Motion repository: MIT license, `motion/react` import
- GSAP standard no-charge license
- Webflow: GSAP becomes free
- react-pdf README: PDF.js worker, cMaps and legacy build with Vite
- React PDF Kit: `?url` worker import and `optimizeDeps` for pdfjs-dist
- Tailwind CSS v4 and shadcn/ui on Vite
- Dexie.js repository
- dexie-react-hooks `useLiveQuery`